Mini RFC: Congestion Control

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

1 Overview

This document specifies a sender-side congestion controller for QUIC similar to TCP NewReno [RFC6582].

The signals QUIC provides for congestion control are generic and are designed to support different sender-side algorithms. A sender can unilaterally choose a different algorithm to use, such as CUBIC [RFC8312].

Related optional features: *QUBIC* (5 pt).

If a sender uses a different controller than that specified in this document, the chosen controller MUST conform to the congestion control guidelines specified in Section 3.1 of [RFC8085].

The algorithm in this document specifies and uses the controller's congestion window in bytes.

2 NewReno

2.1 Bytes in Flight

`bytes_in_flight` is defined as the sum of the size in bytes of all sent packets that contain at least one ack-eliciting or PADDING frame and have not been acknowledged or declared lost. The size does not include IP or UDP overhead, but does include the QUIC header. Packets only containing ACK frames do not count toward `bytes_in_flight` to ensure congestion control does not impede congestion feedback.

`PADDING` frame is not ack-eliciting but is counted in the congestion control.

An endpoint MUST NOT send a packet if it would cause `bytes_in_flight` to be larger than the congestion window, unless when entering recovery.

Packets that include **only** `ACK` frames are not counted in `bytes_in_flight`. If a packet includes both a `STREAM` frame and a `ACK` frame, the size of the entire packet is counted.

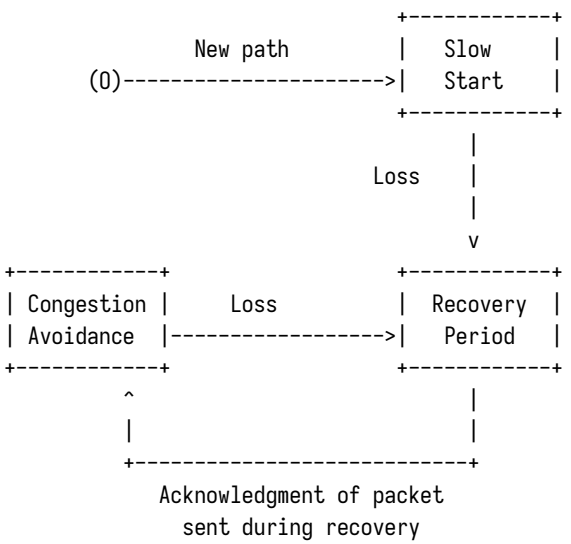
2.2 Initial and Minimum Congestion Window

QUIC begins every connection in slow start with the congestion window set to an initial value. Endpoints SHOULD use an initial congestion window of ten times the maximum datagram size (`max_datagram_size`), while limiting the window to the larger of 14,720 bytes or twice the maximum datagram size. This follows the analysis and recommendations in [RFC6928], increasing the byte limit to account for the smaller 8-byte overhead of UDP compared to the 20-byte overhead for TCP.

The minimum congestion window is the smallest value the congestion window can attain in response to loss. The RECOMMENDED value is 2 * `max_datagram_size`.

2.3 Congestion Control States

The NewReno congestion controller described in this document has three distinct states, as shown in Figure 1.



These states and the transitions between them are described in subsequent sections.

2.3.1 Slow Start

A NewReno sender is in slow start any time the congestion window is below the slow start threshold. A sender begins in slow start because the slow start threshold is initialized to an infinite value.

While a sender is in slow start, the congestion window increases by the number of bytes acknowledged when each acknowledgment is processed. This results in exponential growth of the congestion window.

The sender MUST exit slow start and enter a recovery period when a packet is lost.

2.3.2 Recovery

A NewReno sender enters a recovery period when it detects the loss of a packet. A sender that is already in a recovery period stays in it and does not reenter it.

On entering a recovery period, a sender MUST set the slow start threshold to half the value of the congestion window when loss is detected. The congestion window MUST be set to the reduced value of the slow start threshold before exiting the recovery period.

Implementations MAY reduce the congestion window immediately upon entering a recovery period or use other mechanisms, such as Proportional Rate Reduction [PRR], to reduce the congestion window more gradually. If the congestion window is reduced immediately, a single packet can be sent prior to reduction. This speeds up loss recovery if the data in the lost packet is retransmitted and is similar to TCP.

It's OK to immediately reduce the congestion window. You are free to send a packet before the reduction.

The recovery period aims to limit congestion window reduction to once per round trip. Therefore, during a recovery period, the congestion window does not change in response to new losses.

A recovery period ends and the sender enters congestion avoidance when a packet sent during the recovery period is acknowledged. This is slightly different from TCP's definition of recovery, which ends when the lost segment that started recovery is acknowledged [RFC5681].

2.3.3 Congestion Avoidance

A NewReno sender is in congestion avoidance any time the congestion window is at or above the slow start threshold and not in a recovery period.

A sender in congestion avoidance uses an Additive Increase Multiplicative Decrease (AIMD) approach that MUST limit the increase to the congestion window to at most one maximum datagram size for each congestion window that is acknowledged.

The sender exits congestion avoidance and enters a recovery period when a packet is lost.

2.4 Underutilizing the Congestion Window

When bytes in flight is smaller than the congestion window, the congestion window is underutilized. This can happen due to insufficient application data or flow control limits. When this occurs, the congestion window SHOULD NOT be increased in either slow start or congestion avoidance.

The logic of congestion control is "trial and error"; An endpoint increases the sending speed until it notices that the network could not handle the traffic. When the traffic is small, there is no way to tell whether the network could handle more.